DAY 180/365 · SVM · PART 4 OF 4

SVM in Python: code, results and trade-offs

Run on the breast cancer dataset, with real numbers.

Suyog Yadav

SWIPE →

The article's code, line by line

```
# 1. load the data
cancer = load_breast_cancer()
X = cancer.data[:, :2]
y = cancer.target

# 2. build and train the SVM
svm = SVC(kernel="linear", C=1)
svm.fit(X, y)
```

The task

Tell **benign** from **malignant** tumours. 569 rows, 30 measurements each, 357 benign and 212 malignant.

The settings

`kernel="linear"` means a straight boundary (Part 3). `C=1` is the strictness dial (Part 2).

The article then draws the decision boundary on a scatter plot. That's why it uses just two columns: so the result fits on a 2D picture.

What that code doesn't tell you

Three gaps

1. It uses only **2 of the 30** features (mean radius and mean texture).
2. It never splits into train and test.
3. It never prints a score.

The plot looks convincing, but it doesn't say how good the model is. I scored it:

`svm.score(X, y)` gives **88.9%**, and that's on the **same rows it trained on**. On unseen data, it can only be the same or worse.

Same lesson as my Day 175 post: a pretty result isn't proof until it's tested on data the model hasn't seen.

Numbers from my own run, scikit-learn 1.9.1. A different version may differ slightly.

The proper version

```
X, y = cancer.data, cancer.target  # all 30 features

X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.2,
    random_state=42, stratify=y)

model = make_pipeline(
    StandardScaler(),
    SVC(kernel="rbf", C=1))
model.fit(X_tr, y_tr)
model.score(X_te, y_te)
```

Three changes

All features, a held-out 20% test set, and scaling **inside a pipeline**, so the scaler learns only from training rows (the same leakage rule from my Day 160 post).

What changed in the results

Setup	Accuracy on 114 unseen rows
2 features, linear	**86.8%**
30 features, linear, scaled	**97.4%**
30 features, RBF, scaled	**98.3%**

Using all the features lifts accuracy by about **11 points**. The 2D picture was hiding most of the signal.

> **Don't over-read 97.4 vs 98.3**
>
> With 114 test rows, one row is about 0.9 points. A 5-fold cross-validation puts them level: linear **97.2%**, RBF **97.4%**. For this data, a straight boundary is already enough.

Practice dataset, not a diagnostic tool. My run, random_state=42.

Scaling isn't optional for SVM

Same RBF model, same 5-fold cross-validation. Only one thing differs:

RBF SVM	Mean accuracy
Without scaling	91.2%
With StandardScaler	97.4%

Why

Measurements like **area** (hundreds to thousands) dwarf ones like **smoothness** (about 0.1). Distance-based kernels then listen almost only to the big columns.

That's a **6-point gain from one line of code**, with no change to the model. Remember my Day 161 standardize toggle? Same idea.

Advantages and limitations

Advantages

Many features: copes well in high dimensions, such as images and gene data.
Curves: kernels handle non-linear patterns.
Outliers: soft margin allows a few mistakes.
Multiclass: works for more than two classes.
Memory-light: keeps only support vectors. My RBF model kept **97 of 455** training rows.

Limitations

Slow on large datasets.
Tuning: kernel, C and gamma need care.
Noise: overlapping classes hurt it.
Hard to explain beyond two dimensions.
Needs scaling, as slide 6 showed.

SERIES RECAP

SVM in 4 posts

Day 177: the widest road, with support vectors on its edge.

Day 178: soft margin and the C dial allow a few mistakes.

Day 179: kernels lift data so a line can separate it.

Day 180: in code, use all features, hold out test data and scale in a pipeline.

Would you pick SVM, a random forest or logistic regression for a small, clean classification dataset?